Detección de problemas de seguridade en aplicacións para dispositivos móbiles

Introdución **1**
Mobilidade empresarial **1**
BYOD (Bring Your Own Device) 2
CYOD (Choose Your Own Device) 2
COPE (Corporate-Owned, Personally-Enabled) 3
COBO (Corporate-Owned, Business-Only) 3
OWASP **3**
Modelos de permisos en plataformas móbiles **5**
Permisos 8
Referencias **11**

Introdución

Os dispositivos móbiles son, hoxe en día, elementos indispensables, tanto no ámbito persoal coma no profesional. O seu uso alcanzou un grande nivel desprazando aos ordenadores como ferramenta informática en moitas contornas e resulta moi difícil imaxinar a realización das actividades cotiás sen facer uso deles. A utilización permanente e extensiva destas tecnoloxías converteu aos móbiles nun dos principais obxectivos dos ciberataques nos últimos anos.

A ciberdelincuencia actual volveuse máis sofisticada e as aplicacións móbiles son susceptibles a moitas vulnerabilidades, incluíndo o almacenamento de datos inseguro, controis débiles do lado do servidor e o uso inadecuado da plataforma. Outros problemas comúns soen ser os permisos excesivos do dispositivo, o uso de autenticación débil e unha criptografía insuficiente. Estas vulnerabilidades poden provocar violacións de datos, accesos non autorizados e outros incidentes de seguridade, polo que detectar e mitigar estes problemas de seguridade nas aplicacións móbiles é crucial para protexer os datos das persoas usuarias.

Cando se fala de dispositivos móbiles, as catro categorías de ameazas máis habituais son:

- O compromiso dos dispositivos.
- Os ataques de rede.
- O phising.
- O contido e as aplicacións maliciosas.

O malware e as APIs de terceiros inseguras tamén son ameazas importantes para a seguridade das aplicacións móbiles. A fuga de datos, que ocorre cando se transfire información confidencial sen autorización, é outro problema crítico.

Garantir unha comunicación segura entre a aplicación móbil e o seu servidor é esencial para evitar ataques de intermediarios e outras formas de interceptación de datos.

Mobilidade empresarial

MDM (Mobile Device Management) é unha forma de que as empresas xestionen os dispositivos que se conectan ás súas redes, incluídos smartphones, tabletas, portátiles e outros dispositivos.

MDM permite á empresa administrar, controlar e protexer os dispositivos dende un sistema centralizado. O dispositivo rexístrase no sistema MDM e instálase un perfil de xestión engadindo unha configuración especial ao sistema (Android, iOS, etc.). Este perfil permite recibir ordes do administrador e aplicar políticas de seguridade. Así, a empresa poderá definir normas (uso de PIN obrigatorio, cifrado), decidir que aplicacións se instalan, controlar accesos, bloquear funcións (cámara, USB, etc.), configurar WiFi, etc.

O MDM comproba constantemente o estado do dispositivo podendo notificar ou bloquear o acceso a datos da empresa se non detecta comportamentos sospeitosos.

No caso de que o persoal de administración detecte problemas poderá:

- bloquear o dispositivo.
- localizalo.
- borrar datos de forma remota.
- restablecelo aos valores de fábrica.

Existen catro modelos principais de MDM:

- BYOD (Bring Your Own Device).
- CYOD (Choose Your Own Device)
- COPE (Corporate-Owned, Personally-Enabled)
- COBO (Corporate-Owned, Business-Only).

Cada modelo ten as súas propias vantaxes e inconvenientes, e o modelo adecuado para unha empresa depende das necesidades desta.

BYOD (Bring Your Own Device)

BYOD, ou “trae o teu propio dispositivo” é o modelo MDM máis popular. Permite ás persoas traballadoras utilizar os seus propios dispositivos persoais para fins laborais, o que pode aportar vantaxes como o aforro de custos (xa que a empresa non ten que comprar dispositivos para o persoal) e unha maior satisfacción das persoas traballadoras (xa que xa están familiarizadas cos seus propios dispositivos e como utilizalos).

Instálase un perfil MDM no dispositivo só para os datos corporativos, que se manterán separados dos datos persoais.

Sen embargo, tamén implica algúns riscos, xa que non hai forma de que a empresa controle de forma global o dispositivo, o que pode provocar fendas de seguridade e fugas de datos, así como problemas de compatibilidade.

Isto pode supoñer un reto para os departamentos informáticos, que deben xestionar e protexer estes dispositivos ao mesmo tempo que manteñen a privacidade das persoas traballadoras.

CYOD (Choose Your Own Device)

CYOD, ou “escolle o teu propio dispositivo”, é un modelo no que a empresa proporciona unha lista de dispositivos aprobados entre os que as persoas traballadoras poden escoller. Deste modo, a empresa ten máis control sobre os dispositivos que se utilizan e pode evitar problemas de compatibilidade e seguridade.

As desvantaxes de CYOD son que pode resultar máis caro para a empresa, pode ser difícil manterse ao día cos últimos dispositivos e sistemas operativos, e as persoas traballadoras poden estar menos comprometidas se non están utilizando os seus propios dispositivos persoais.

COPE (Corporate-Owned, Personally-Enabled)

COPE ou “propiedade da empresa, habilitado para uso persoal” é un modelo no que a empresa proporciona dispositivos ás persoas traballadoras para que os utilicen no traballo, pero tamén poden ser utilizados con fins persoais, dentro de límites. A vantaxe de COPE é a facilidade para xestionar e protexer os dispositivos.

A desvantaxe da COPE é que pode resultar máis cara para a empresa e que as persoas traballadoras poden estar menos satisfeitas cos dispositivos que se lles entregan.

COBO (Corporate-Owned, Business-Only)

No modelo COBO “propiedade da empresa, só para uso empresarial”, a empresa proporciona ás persoas traballadoras dispositivos de propiedade corporativa que só poden utilizarse con fins laborais. Este é o modelo máis seguro, xa que axuda a manter separados os datos laborais dos persoais e garante que os dispositivos só se utilicen para actividades relacionadas co traballo.

Sen embargo, pode resultar caro proporcionar dispositivos a cada persoa traballadora, e algunhas persoas poden preferir utilizar os seus propios dispositivos.

OWASP

Ao igual que para o caso do desenvolvemento web seguro, OWASP xoga un papel moi relevante no desenvolvemento móbil seguro.

O proxecto de seguridade de aplicacións móbiles de OWASP (MAS, Mobile Application Security) é un proxecto creado por OWASP para axudar ás persoas desenvolvedoras e profesionais de seguridade a comprender e mitigar os riscos particulares de seguridade asociados ás aplicacións móbiles. O proxecto tamén inclúe un checklist para axudar ás organizacións a probar a seguridade das súas aplicacións móbiles.

O proxecto MAS de OWASP proporciona:

- OWASP MASVS - Mobile Application Security Verification Standard: un estándar de seguridade para aplicacións móbiles que describe os requisitos de seguridade para as aplicacións móbiles.
- OWASP MASWE - Mobile Application Security Weakness Enumeration: unha lista de debilidades de seguridade e privacidade comúns específicas de aplicacións móbiles.
- OWASP MASTG - Mobile Application Security Testing Guide: unha guía de probas de seguridade e enxeñería inversa. Ofrece instrucións detalladas para probar os requisitos de seguridade de aplicacións móbiles. Describe os procesos técnicos para verificar os controis enumerados en OWASP MASVS a través das debilidades definidas por OWASP MASWE.
- OWASP MAS Checklist: contén ligazóns aos casos de probas de OWASP MASTG para cada control OWASP MASVS.

Estes recursos son recoñecidos a nivel internacional como estándares da industria para a seguridade de aplicacións móbiles.

O OWASP Mobile Top 10 é unha lista que identifica os riscos de seguridade máis comúns en aplicacións móbiles. Esta lista é mantida pola OWASP Foundation. Na versión de 2024 incluíanse os seguintes riscos:

- **M1: Uso inadecuado de credenciais:** a xestión insegura de credenciais ocorre cando as aplicacións móbiles utilizan credenciais codificadas directamente no código ou cando se fai un uso indebido delas, como a súa transmisión de forma insegura, o almacenamento inseguro ou a autenticación débil de usuarios na aplicación.
- **M2: Seguridade inadecuada na cadea de subministro:** prodúcese debido á falta de prácticas de codificación seguras, a revisións e probas de código insuficientes. Tamén pode producirse debido a un proceso de sinatura e distribución de aplicacións insuficiente ou inseguro, debilidades nos compoñentes ou bibliotecas de software de terceiros, controis de seguridade insuficientes para os datos, o cifrado e o almacenamento ou a exposición de datos confidenciais a acceso non autorizado.
- **M3: Autenticación/autorización insegura:** Comprender a diferenza entre autenticación e autorización é fundamental para avaliar a seguridade das aplicacións móbiles. A **autenticación** identifica a un individuo, mentres que a **autorización** verifica se o individuo identificado ten os permisos necesarios para unha acción particular. Estes dous aspectos están estreitamente relacionados, xa que as comprobacións de autorización deben seguir inmediatamente á autenticación da solicitude do dispositivo móbil. Nas aplicacións móbiles son necesarios mecanismos de autenticación e autorización robustos e seguros, tanto no control de acceso do usuario como nos recursos sobre os que ten acceso, evitando accesos non autorizados.
- **M4: Validación insuficiente de entrada/saída:** ocorre cando unha aplicación non consegue comprobar e sanear correctamente a entrada do usuario ou validar e sanear os datos de saída. É importante validar tanto os datos de entrada como os de saída para previr ataques de inxección de código, inxección SQL, XSS, acceso non autorizado ao sistema, entre outros.
- **M5: Comunicación insegura:** a transmisión de información sensible a través de canais non seguros, cando se transmiten en texto plano ou sen un bo protocolo de cifrado, deixa expostas as aplicacións móbiles a unha batería de ataques que poderían permitir a interceptación das comunicacións e a súa manipulación.
- **M6: Controis de privacidade inadecuados:** case todas as aplicacións procesan algún tipo de información persoal identificable. Unha aplicación é vulnerable a violacións da privacidade se un atacante pode acceder a eses datos a través dun medio de almacenamento ou transmisión que non está suficientemente seguro.
- **M7: Protecciónns binarias insuficientes**: un ficheiro binario de aplicación podería ser obxecto de dous tipos de ataque:
    - Enxeñería inversa: o binario descompílase e analízase en busca de información valiosa, como claves secretas, algoritmos ou vulnerabilidades.
    - Manipulación de código: o binario da aplicación manipúlase, por exemplo, para eliminar comprobacións de licenza, burlar muros de pago ou obter outros beneficios como usuario. Alternativamente, a aplicación pode manipularse para conter código malicioso.

  A unión da “enxeñería inversa” e a “manipulación de código” pode converterse nun problema de seguridade nas aplicacións móbiles.

En xeral, as aplicacións completamente compiladas como as aplicacións de iOS son menos susceptibles á enxeñaría inversa e á manipulación do código que o bytecode de nivel superior que se atopa nas aplicacións de Android (isto pode non ser válido para aplicacións desenvolvidas con tecnoloxías multiplataforma, como PWA - Progressive Web Apps ou Flutter).

- **M8: Configuración de seguridade incorrecta:** a configuración incorrecta de seguranza nas aplicacións móbiles refírese á configuración incorrecta dos axustes, permisos e controis de seguranza que pode levar a vulnerabilidades e acceso non autorizado.
- **M9: Almacenamento de datos inseguro**: ocorre cando unha aplicación utiliza un cifrado débil, mecanismos de almacenamento de datos inseguros (como ficheiros de texto sen formato ou bases de datos desprotexidas), protocolos de transmisión de datos inseguro ou unha xestión inadecuada das credenciais dos usuarios.
- **M10: Criptografía insuficiente:** o uso inadecuado ou incorrecto de algoritmos criptográficos pode levar a comprometer a confidencialidade e a integridade dos datos sensibles. Ocorre cando se usan algoritmos de cifrado débiles ou lonxitudes de clave inadecuadas, prácticas de xestión de claves deficientes, manexo inadecuado das claves de cifrado, xeración insegura de números aleatorios, implementación defectuosa de protocolos criptográficos ou vulnerabilidades en bibliotecas ou marcos criptográficos.

A partir destes riscos poden extraerse estratexias que deben implementar os equipos de desenvolvemento para mellorar a seguridade das súas aplicacións móbiles e protexer os datos das persoas usuarias de posibles ameazas.

Modelos de permisos en plataformas móbiles

Os permisos das aplicacións móbiles son un aspecto chave tanto no proceso de desenvolvemento como no seu posterior uso. Estes permisos determinan como e en que contexto a aplicación accederá a certas funcións e datos específicos do dispositivo.

Conceder permisos innecesarios ou incorrectos a unha aplicación pode provocar vulnerabilidades de seguridade e que, abusando destes permisos, aplicacións maliciosas accedan a datos confidenciais ou realicen certas accións non autorizadas, que son actualmente algunhas das ameazas máis importantes para a seguridade nas plataformas móbiles.

As plataformas móbiles utilizan modelos de permisos para administrar e controlar o acceso a datos confidenciais e recursos do sistema co fin de manter a privacidade e a seguridade das persoas usuarias.

Na plataforma Android os permisos das aplicacións móbiles poden clasificarse en tres categorías:

- **Permisos no momento da instalación (install-time permissions)**. Antes de Andoid 6.0 (Marshmallow) todos os permisos se aceptaban ao instalar a aplicación. Se non se concedían, non se podía instalar. Dende Android 6.0 introdúcense os

permisos en tempo de execución, é dicir, a aplicación pide permiso cando o necesita.

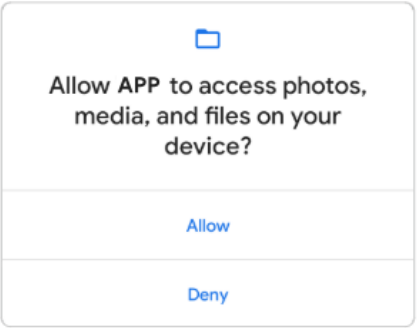
Os permisos de instalación son outorgados polo sistema cando a aplicación se instala. Este tipo de permisos, que non representan un perigo para a privacidade das persoas usuarias nin para o funcionamento do dispositivo, considéranse de baixo risco. Son os permisos mínimos necesarios para que a aplicación funcione.

Android inclúe varios subtipos de permisos no momento da instalación:

- **Permisos normais:** son permisos que presentan un risco mínimo para a privacidade e o funcionamento doutras aplicacións. O sistema outorga automaticamente este tipo de permiso a unha aplicación solicitante durante a instalación, sen pedir a aprobación explícita da persoa usuaria, aínda que a persoa usuaria sempre ten a opción de revisar estes permisos antes da instalación. Exemplo: acceso a internet.
- **Permisos de firma:** é un permiso que o sistema concede unicamente se a aplicación solicitante está asinada co mesmo certificado que a aplicación que declarou o permiso. Se os certificados coinciden, o sistema concede automaticamente o permiso sen notificar á persoa usuaria nin solicitar a súa aprobación explícita.

  Dentro dos permisos de firma, recentemente apareceu o permiso de **asinante coñecido (knownSigner)**, que é un permiso que o sistema concede unicamente se a aplicación solicitante está asinada con un certificado coñecido e de confianza para o sistema. Se o certificado do solicitante está na lista, o sistema concede automaticamente o permiso sen notificar á persoa usuaria nin solicitar a súa aprobación explícita.

- **Permisos de execución (runtime permissions)**. Tamén son coñecidos como **permisos perigosos**. Estes permisos solicítanse durante o funcionamento da aplicación e requiren o consentimento explícito da persoa usuaria. Outorgan á aplicación solicitante acceso a datos privados da persoa usuaria ou control sobre o dispositivo. O desexable é que as persoas usuarias poidan controlar a que datos confidenciais acceden as aplicacións. Este tipo de permisos son os que se consideran máis perigosos.



- **Permisos especiais**. Os permisos especiais utilízanse cando unha aplicación necesita solicitar operacións privilexiadas máis alá dos permisos de tempo de execución normais. Estes permisos utilízanse para casos especiais, como modificar

a configuración do sistema ou acceder a funcións de hardware específicas. Só a plataforma e os fabricantes de equipos orixinais (OEM) poden definir permisos especiais.

A configuración dos permisos dunha aplicación móbil de Android realízase no ficheiro **AndroidManifest.xml**. Este arquivo de manifesto describe información esencial sobre a aplicación, necesaria para o sistema operativo e o instalador, e importante para que a aplicación funcione correctamente.

Entre outras cousas, o ficheiro AndroidManifest.xml debe declarar o seguinte:

- Os compoñentes da aplicación, incluídas todas as **activities, services, broadcast receivers** e **content providers.** Cada compoñente debe definir propiedades básicas, como o nome da súa clase.
- Os permisos que necesita a aplicación para acceder a partes protexidas do sistema ou a outras aplicacións. Tamén declara os permisos que deben ter outras aplicacións se queren acceder ao contido desta aplicación.
- As características de hardware e software que require a aplicación, como unha cámara, servizos Bluetooth ou unha pantalla multitáctil.
- Declara o nivel de API mínimo requirido pola aplicación.
- Declara as bibliotecas API coas que a aplicación debe vincularse

O arquivo XML pode xerarse manualmente, o que en moitos casos pode resultar bastante laborioso. Se se utiliza Android Studio para crear unha aplicación, o arquivo de manifesto créase automaticamente e a maioría dos elementos esenciais do manifesto agréganse a medida que se crea a aplicación, especialmente cando se usan plantillas de código.

O seguinte XML é un exemplo simple de **AndroidManifest.xml** que declara dúas actividades para a aplicación.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:versionCode="1"
   android:versionName="1.0">

   <!-- Beware that these values are overridden by the build.gradle file -->
   <uses-sdk android:minSdkVersion="15" android:targetSdkVersion="26" />
```

```
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <!-- This name is resolved to com.example.myapp.MainActivity
             based on the namespace property in the build.gradle file -->
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity
            android:name=".DisplayMessageActivity"
            android:parentActivityName=".MainActivity" />
    </application>
</manifest>
```

Permisos

As aplicacións de Android deben solicitar permiso para acceder a datos confidenciais da persoa usuaria, como contactos e SMS, ou a determinados recursos e funcións do dispositivo, como a localización, a cámara, o micrófono, os contactos, a galería de imaxes e vídeos e as comunicacións coa conexión wifi e bluetooth.

Tanto en plataformas Android como en plataformas iOS as persoas usuarias poden administrar a posteriori os permisos asignados ás aplicacións nos axustes do sistema do dispositivo

Cada permiso identifícase mediante unha etiqueta única. Por exemplo, unha aplicación que necesita enviar mensaxes SMS debe ter a seguinte liña no arquivo de manifesto.

```
<manifest ... >
    <uses-permission android:name="android.permission.SEND_SMS"/>
    ...
</manifest>
```

A partir de Android 6.0 (nivel de API 23), as persoas usuarias deben aprobar ou rexeitar algúns permisos da aplicación en tempo de execución. Pero, independentemente da versión de Android que admita a aplicación, deben declararse todas as solicitudes de permiso con un elemento <uses-permission> no arquivo AndroidManifest.xml. Se se concede o permiso, a aplicación poderá usar as funcións protexidas. En caso contrario, os seus intentos de acceder a esas funcións fallarán.

Cada permiso ten un **nivel de protección** asociado, que se refire ao risco potencial implícito no permiso. Deste tipo depende o procedemento que debe seguir o sistema para determinar se outorga ou non dito permiso a unha aplicación que o solicita.:

Exemplos de permisos de aplicacións Android:

- **android.permission.INTERNET**: permite que as aplicacións abran sockets de rede. Nivel de protección: **normal**
- **android.permission.ACCESS_NETWORK_STATE**: permite que as aplicacións accedan a información sobre as redes. Nivel de protección: **normal**
- **android.permission.ACCESS_FINE_LOCATION**: permite que unha aplicación acceda a unha ubicación precisa. Nivel de protección: **perigoso**
- **android.permission.CALL_PHONE**: permite que unha aplicación inicie unha chamada telefónica sen pasar pola interface de usuario do Marcador para que a persoa usuaria confirme a chamada. Nivel de protección: **perigoso**
- **android.permission.CAMERA**: permiso necesario para poder acceder ao dispositivo de cámara. Nivel de protección: **perigoso**
- **android.permission.BATTERY_STATS**: permite que unha aplicación recopile estatísticas da batería. Nivel de protección: **sinatura**

Listado de permisos de Android: Manifest.permission | API reference | Android Developers

Os permisos da aplicación decláranse no arquivo AndroidManifest.xml mediante a etiqueta <uses-permission>.

Cando unha aplicación vaia usar algunhas das funcións ou recursos para os que pediu permiso, deberá comprobar se se lle concedeu ese permiso. Por exemplo, se se quere utilizar o permiso para acceder a información sobre a rede, podería escribirse o seguinte bloque de código Java:

```
if (ContextCompat.checkSelfPermission(this, Manifest.permission
ACCESS_NETWORK_STATE) != PackageManager.PERMISSION_GRANTED) {
    // Se o permiso non se concedeu, hai que solicitalo
    ActivityCompat.requestPermissions(this, new
String[](Manifest.permission.ACCESS_NETWORK_STATE), 1);
}
```

Para comprobar se se outorgou permiso á aplicación Android para acceder á información da rede utilízase o método **checkSelfPermission** da clase ContextCompat, que devolve o estado actual dese permiso, neste caso o permiso é Manifest.permission.ACCESS_NETWORK_STATE.

Para determinar se se lle outorgou el permiso pode compararse o resultado con PackageManager.PERMISSION_GRANTED.

Se non se outorgou ese permiso, chámase ao método **requestPermissions** da clase ActivityCompat para solicitar o permiso necesario á persoa usuaria.

NOTA: Dende Android 6.0 (API 23) ás persoas usuarias non se lles solicitan permisos no momento da instalación, polo que os equipos de desenvolvemento deben solicitar aqueles permisos incluídos no arquivo de manifesto en tempo de execución.

Actualmente, cando se solicitan os permisos ás persoas usuarias é habitual que non só se ofrezan as opcións do tipo "Si" e “Non”, senón que tamén se informa e se pregunta como se quere conceder ese permiso, e a persoa usuaria pode decidir entre diferentes opcións, como poden ser “Mentres se usa a aplicación”, “Só esta vez” ou “Sempre”. Se a persoa usuaria opta por “Só esta vez”, só se concede o permiso para ese momento e uso concreto, polo que, se a aplicación volve a requirir ese permiso noutra ocasión, terá que volver a pedir confirmación. O permiso “Mentres se usa a aplicación” permite que a aplicación acceda á función ou aos datos só cando a aplicación está en primeiro pano, e, polo contrario a opción “Sempre” concede o permiso á aplicación incluso cando esta se executa en segundo plano.

Á hora de implementar a xestión dos permisos da aplicación, o equipo responsable de desenvolvemento debe contemplar unha serie de **boas prácticas recomendadas**:

- **Solicitar permisos mínimos**: unha das recomendacións clave é solicitar só os permisos que sexan absolutamente necesarios para que a aplicación funcione. Os equipos de desenvolvemento deben considerar que permisos son realmente necesarios, eliminando o resto. Desta maneira redúcese a superficie de ataque e limítanse as posibilidades de que a aplicación se use indebidamente.
- **Informar do propósito dos permisos ás persoas usuarias**: proporcionar razóns claras e descritivas ás persoas usuarias sobre por que se solicita un permiso e o valor que aportará á súa experiencia na aplicación xerará máis confianza e maiores posibilidades de que se conceda.
- **Optimizar cando se solicitan os permisos**: recoméndase que os permisos se soliciten no momento adecuado, como cando a persoa usuaria está a punto de usar a funcionalidade que require o permiso. Abusar con solicitudes de permisos inmediatamente tras instalar ou iniciar unha aplicación pode minar a confianza da persoa usuaria facendo que os denegue.

Á hora de controlar se unha aplicación pode facer uso dunha funcionalidade ou recurso do dispositivo móbil, non só está involucrada a persoa usuaria, xa que os sistemas operativos móbiles dispoñen de mecanismos para protexer o sistema de acceso non autorizado e do uso indebido por parte de aplicacións móbiles.

Os dispositivos móbiles dispoñen dunha capa de abstracción de hardware (**HAL**, Hardware Abstraction Layer) que comunica o núcleo e as capas de software superiores do sistema operativo móbil. A capa HAL oculta os detalles específicos do hardware creando unha forma estándar de traballar cos compoñentes de hardware do dispositivo móbil. Esta estandarización permite que as aplicacións interactúen co hardware do dispositivo sen necesidade de comprender os detalles específicos de como funciona.

Os sistemas operativos móbiles proporcionan ás aplicacións móbiles as denominadas APIs (Application Programming Interfaces) para acceder aos recursos e servizos do sistema. Para desenvolver aplicacións que traballen con estas APIs utilízanse frameworks para aplicacións móbiles; entre os máis coñecidos inclúense SDK de Android e UIKit de iOS.

Estes mesmos frameworks proporcionan un conxunto de APIs que permiten aos equipos de desenvolvemento acceder aos recursos do sistema e ás funcións do hardware.

Por exemplo, se unha aplicación de Android necesita usar a cámara do dispositivo para facer unha foto, a través da API para a cámara que proporciona SDK os equipos de desenvolvemento poden facer que a súa aplicación interactúe co hardware da cámara. A aplicación usaría unha chamada do sistema para solicitar acceso ao hardware da cámara e o sistema operativo móbil verificaría se a aplicación ten os permisos necesarios para acceder á cámara. En caso afirmativo o sistema operativo outorgaría acceso e a aplicación podería usar a API da cámara para controlar a cámara e facer a foto.

Para protexer as chamadas ao sistema, Android dispón de **Seccomp (secure computing mode)**, una función do kernel de Linux implementada desde Android 8 (Oreo) que é un mecanismo de seguridade do kernel de Linux que permite limitar as chamadas ao sistema que pode facer un proceso. Unha das tarefas de Seccomp é filtrar as chamadas ao sistema, é dicir, comproba se é seguro que unha aplicación faga unha chamada concreta ao sistema e, en caso contrario, será bloqueada. Ao restrinxir as chamadas ao sistema que unha aplicación pode realizar, se un atacante logra explotar unha vulnerabilidade da aplicación, Seccomp limitaría o dano potencial que podería causarse a través da aplicación comprometida.

Referencias

Para a elaboración deste material utilizáronse, entre outros, os recursos que se enumeran a continuación:

- Curso F2101037 Posta en produción segura
- INCIBE
- Fernández Riera, Máximo (2022) Puesta en producción segura. Ra-Ma.
- Romero, Ternero, Mª Carmen (2024) Puesta en producción segura. Paraninfo.
- Qué modelo de MDM elegir: BYOD, CYOD, COPE y COBO